ABSTRACT

WHAT IF? – Interactive Data Simulation and Comparison Lab is a Python-based desktop application developed to help users explore datasets through controlled hypothetical modifications.

It provides a graphical environment in which users can load CSV files, inspect records and descriptive statistics, apply arithmetic or conditional operations, create named scenarios, and compare modified data with an original dataset or another CSV file. The objective is to make basic data experimentation more accessible without requiring users to edit files manually or calculate every difference independently.

The application uses Tkinter for its graphical user interface, Pandas for tabular data management, NumPy for numerical operations, and Matplotlib for visualisation. CSV is used for dataset input and output, while JSON is used to maintain experiment-history metadata. Its five main functional areas are Dashboard, Dataset Explorer, Experiment Lab, Comparison Studio, and Insights & Reports.

The Experiment Lab supports percentage increase and decrease, addition, subtraction, multiplication, and row filtering through matching or removal conditions. Users may optionally define conditions using equals, not equals, greater than, less than, and contains.

The Comparison Studio identifies common numerical columns, calculates mean values, determines absolute and percentage changes where mathematically valid, and presents a grouped bar chart. The reporting component provides descriptive observations and export options.

This is an educational and exploratory tool. Its results describe the changes produced by user-defined operations and should not be interpreted as predictions or proof of causal relationships.

Keywords: Python, Data Simulation, Pandas, Tkinter, NumPy, Matplotlib, CSV, Scenario Analysis, Data Visualisation.

1. INTRODUCTION

1.1 Background

Data analysis often involves asking how a dataset would look if selected values or records were changed. Such questions are useful in learning, planning, quality checks, and exploratory analysis. A user may want to increase a numerical column by a fixed percentage, remove records that do not meet a condition, or compare two versions of a dataset. Performing these tasks manually can involve repeated editing, calculations, and chart preparation.

1.2 Project Overview

WHAT IF? provides a single desktop workspace for these activities. The user loads a CSV dataset, explores its structure, selects an operation, defines the target column and optional condition, and creates a named scenario. The scenario can then be compared with the original dataset or with a second CSV file. The application calculates descriptive differences and displays a visual comparison.

1.3 Problem Statement

A common difficulty in introductory data analysis is the fragmented nature of simple experimentation. Dataset inspection, modification, calculation, comparison, charting, and exporting may require separate tools or manual steps. This project addresses that inconvenience by combining these tasks in one graphical application.

1.4 Motivation

The project was motivated by the value of interactive learning. A user can change one variable at a time, observe how the resulting values differ, and compare multiple named scenarios. This makes data-processing operations easier to understand and demonstrates practical uses of Python libraries.

1.5 Nature of the Application

The application performs deterministic transformations based on values and conditions entered by the user. It is not a machine-learning system and does not forecast future values. The term simulation refers to applying hypothetical changes to a copy of the dataset and examining the resulting descriptive statistics.

2. OBJECTIVES AND SCOPE

2.1 Objectives

- Develop a desktop application using Python and Tkinter.
- Allow users to import CSV datasets and inspect records.
- Display basic descriptive statistics for numerical data.
- Support arithmetic modifications and conditional row filtering.
- Create named scenarios based on a consistent original dataset.
- Compare datasets using common numerical columns.
- Calculate absolute and percentage changes in mean values where possible.
- Display grouped bar charts and descriptive observations.
- Export modified datasets and text-based reports.
- Demonstrate practical integration of data processing and visualisation libraries.

2.2 Scope

The current scope covers local, individual use with CSV datasets. It supports data exploration, simple numerical transformations, condition-based filtering, scenario comparison, charts, and file export. The supplied demonstration dataset contains 12 student-related records with columns such as Study_Hours, Attendance, and Marks. This dataset is illustrative; the application is intended to accept other suitable CSV files as well.

2.3 Out of Scope

- Predictive modelling and machine learning.
- Real-time data acquisition or cloud synchronisation.
- Multi-user collaboration and access control.
- Advanced hypothesis testing or causal inference.
- Automatic recommendations or decision-making.

2.4 Intended Users

The intended users include students, beginners learning data analysis, and individuals who need to explore simple tabular datasets. Users should have basic familiarity with CSV files and numerical columns.

3. EXISTING AND PROPOSED SYSTEM

3.1 Existing Approach

Without a dedicated experimentation interface, a user may open a CSV file in a spreadsheet or write separate scripts to inspect, modify, and compare data. The user may need to maintain copies of the original and modified files, remember which operations were applied, calculate summary statistics, and create charts independently.

3.2 Limitations

- Repeated manual editing can introduce accidental changes to source data.
- Separate tools for calculations and charts can interrupt the workflow.
- Comparing several hypothetical situations may require duplicate files and careful naming.
- Manual percentage calculations can be inconsistent or error-prone.
- The process may be difficult for beginners learning data-processing libraries.

3.3 Proposed System

WHAT IF? combines dataset exploration, controlled modifications, scenario creation, statistical comparison, visualisation, and export in one Python desktop application. The original dataset is retained as the baseline for scenario creation. This makes each experiment easier to compare against a consistent reference.

Table 1. Existing and Proposed System Comparison

Aspect	Manual / Separate Approach	WHAT IF? Proposed Approach
Dataset inspection	Spreadsheet or script	Dataset Explorer preview and summaries
Modification	Manual editing or separate code	Experiment Lab operations
Scenario tracking	Separate files	Named scenarios and JSON metadata
Comparison	Manual calculation	Automatic mean and change calculations
Visualisation	Created separately	Grouped chart in Comparison Studio
Export	Manual save process	Application export functions

4. REQUIREMENT ANALYSIS

4.1 Functional Requirements

Functional requirements describe the services the application is expected to provide to its user.

Table 2. Functional Requirements

ID	Requirement	Description
FR-01	Load dataset	Load sample or user-selected CSV.
FR-02	Preview dataset	Display records for inspection.
FR-03	View statistics	Present descriptive statistics for numerical columns.
FR-04	Create experiment	Apply a selected operation to a target column.
FR-05	Conditional operation	Restrict an operation or row filter using a condition.
FR-06	Save scenario	Assign a name and retain scenario metadata.
FR-07	Compare datasets	Compare original data with scenario or second CSV.
FR-08	Visualise comparison	Generate grouped bar chart for common numerical columns.
FR-09	Export data	Save modified dataset as CSV.
FR-10	Export report	Save a text-based summary.

4.2 Non-Functional Requirements

Table 3. Non-Functional Requirements

Requirement	Description
Usability	Controls and navigation should be understandable to beginners.
Reliability	Validate inputs and handle common file/data errors.
Performance	Basic operations on small and medium files should complete practically.
Maintainability	Code should be organised for updates.
Portability	Run with compatible Python and dependencies.
Clarity	Results, labels and messages should be understandable.

4.3 Hardware Requirements

Table 4. Suggested Hardware Requirements

Component	Suggested Minimum
Processor	Dual-core processor
RAM	4 GB
Storage	Approximately 500 MB free
Display	Standard desktop monitor
Input devices	Keyboard and mouse

These are suggested baseline requirements, not results from formal hardware benchmarking.

4.4 Software Requirements

Table 5. Software Requirements

Software	Purpose
Python 3	Runs the application.
Tkinter	Desktop GUI.
Pandas	Tabular data loading and manipulation.
NumPy	Numerical calculations.
Matplotlib	Charts and visual comparisons.
CSV	Dataset input and output.
JSON	Experiment-history metadata.
Visual Studio Code	Development and debugging.

5. TECHNOLOGY STACK

Table 6. Technology Stack

Technology	Application Area	Contribution
Python	Core	Coordinates interface, processing and files.
Tkinter	GUI	Windows, buttons, input controls and navigation.
Pandas	Data handling	Reads CSV into DataFrames; filtering and summaries.
NumPy	Numerical processing	Supports array-based calculations.
Matplotlib	Visualisation	Grouped bar charts.
CSV	Data exchange	Input datasets and exported scenarios.
JSON	Metadata	Experiment-history information.
VS Code	Development	Editor, terminal and debugging.

5.1 Python

Python is the main programming language because it has readable syntax and a broad ecosystem of libraries for desktop interfaces and data analysis. It coordinates user interactions, transformations, calculations, and file operations.

5.2 Tkinter

Tkinter is Python's standard interface to the Tcl/Tk GUI toolkit. It provides frames, labels, buttons, entry fields, menus, and dialog windows. In this project it forms the presentation layer.

5.3 Pandas and NumPy

Pandas provides the DataFrame structure used to represent tabular datasets. Its CSV-reading and writing functions support import and export. DataFrame operations select columns, apply conditions, and calculate descriptive statistics. NumPy supports numerical computation.

5.4 Matplotlib

Matplotlib creates grouped bar charts comparing original and modified mean values. A chart gives the user a visual summary in addition to numerical differences.

5.5 CSV and JSON

CSV is a simple tabular text format suitable for datasets. JSON is a structured text format suitable for metadata such as experiment names and parameters. JSON history metadata is not a relational database.

6. SYSTEM ARCHITECTURE

The application follows a modular desktop architecture. The user interacts with the Tkinter presentation layer. Application logic receives selections and coordinates dataset operations. Pandas and NumPy handle data processing, while experiment and comparison functions perform the analytical tasks. Matplotlib presents comparisons visually. File-handling functions import and export CSV and text files, and JSON stores experiment metadata.

Layer	Responsibility
Presentation	Tkinter screens, forms, controls and messages.
Application logic	Navigation and coordination of user actions.
Data processing	Pandas DataFrames and NumPy operations.
Experiment	Arithmetic operations and conditional filters.
Comparison	Common numerical columns and change calculations.
Visualisation	Matplotlib grouped bar charts.
File handling	CSV input and CSV/text output.
History storage	JSON experiment metadata.

Figure 1. System Architecture

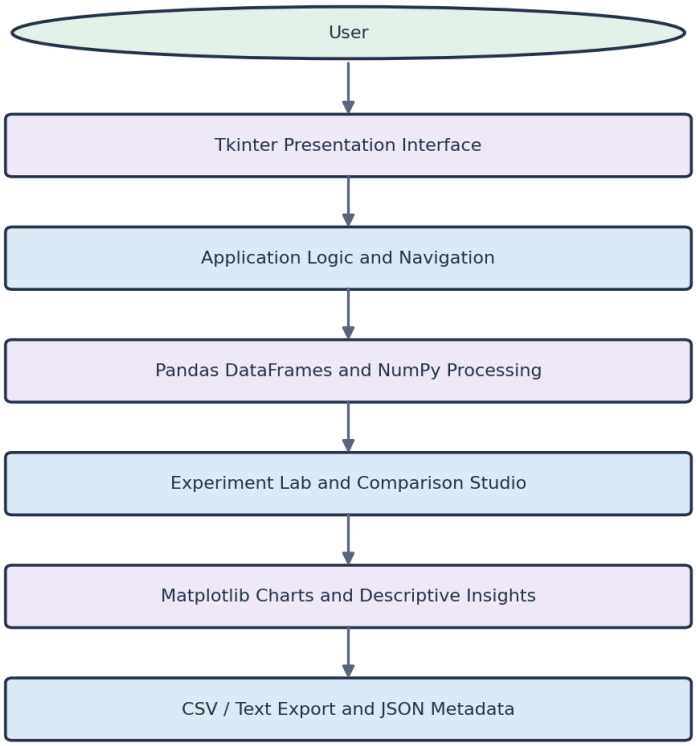


Figure 1. Layered architecture of the WHAT IF? application

Table 7. System Architecture Layers

6.1 Data Movement

A CSV file is loaded into a DataFrame and inspected. When an experiment is submitted, the application validates parameters and works on a copy of the original data. The resulting scenario is available for comparison. The comparison stage calculates summary values and sends them to visualisation and reporting components. Export functions write selected results to files.

7. APPLICATION MODULES

Table 8. Application Modules

Module	Responsibilities	Key Functionalities
Dashboard	Main navigation	Links to working modules.
Dataset Explorer	Dataset inspection	CSV loading, preview and numerical summaries.
Experiment Lab	Hypothetical modification	Arithmetic operations, conditional filters, named scenarios.
Comparison Studio	Compare data versions	Common numeric columns, means, changes, grouped chart.
Insights & Reports	Present and export results	Descriptive observations, CSV and text export.

7.1 Dashboard

The Dashboard acts as the landing page. It provides access to major sections and helps the user move between dataset exploration, experimentation, comparison, and reporting.

7.2 Dataset Explorer

The Dataset Explorer loads a sample or external CSV file and presents its records. It helps identify available columns and inspect descriptive statistics before a target numerical column is selected.

7.3 Experiment Lab

The Experiment Lab is the primary transformation module. The user chooses a target column, operation, input value, and optionally a condition. The application validates inputs and applies the operation to the appropriate values in a copy of the original DataFrame.

7.4 Comparison Studio

The Comparison Studio accepts a saved scenario or second CSV file. It finds common columns and uses common numerical columns for statistical comparison. It calculates means, absolute changes, and percentage changes where the denominator permits.

7.5 Insights & Reports

The reporting module turns calculated values into descriptive observations. It also provides options to export the active modified dataset and a text-based report.

8. EXPERIMENT OPERATIONS

The Experiment Lab provides arithmetic transformations and row-filtering operations. Arithmetic operations affect a selected target column. Filtering operations change which rows remain in the resulting scenario.

Table 9. Supported Experiment Operations

Operation	Description	Example
Increase (%)	Raises selected values by a percentage.	Increase Study_Hours by 20%.
Decrease (%)	Reduces selected values by a percentage.	Decrease Attendance by 10%.
Add	Adds a fixed number.	Add 5 to Marks.
Subtract	Subtracts a fixed number.	Subtract 2 from Study_Hours.
Multiply	Multiplies by a factor.	Multiply Marks by 2.
Keep Matching Rows	Retains records meeting condition.	Keep Marks greater than 40.
Remove Matching Rows	Removes records meeting condition.	Remove Attendance below 50.

8.1 Conditional Operators

Table 10. Conditional Operators

Operator	Meaning	Example
Equals	Value equals condition.	Attendance equals 80.
Not Equals	Value differs from condition.	Section not equals A.
Greater Than	Value is greater.	Marks greater than 60.
Less Than	Value is lower.	Study_Hours less than 2.
Contains	Text includes specified text.	Name contains 'A'.

8.2 Example Experiment

To increase Study_Hours by 20% only for records where Attendance is greater than 75, the user selects Study_Hours, Increase (%), enters 20, selects Attendance as the condition column, chooses Greater Than, and enters 75. The application evaluates the condition and applies the increase to matching records. Other records remain unchanged.

8.3 Scenario Baseline

Each scenario is intended to be derived from the original DataFrame rather than from the previously modified scenario. This gives experiments a common baseline and avoids accidentally compounding changes.

9. EXPERIMENT PROCESSING

Figure 3. Experiment Processing Flowchart

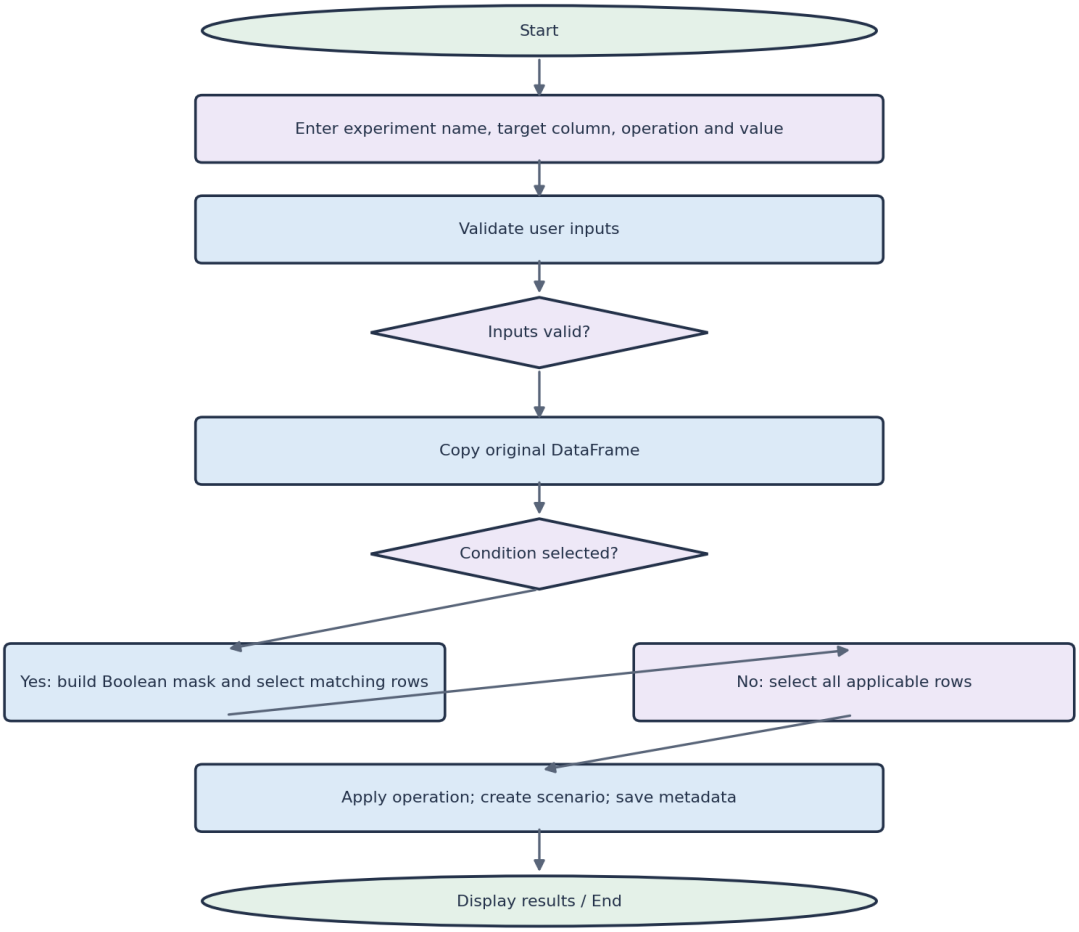


Figure 3. Experiment processing flowchart

9.1 Processing Steps

1. Enter experiment name and select target column.
2. Choose arithmetic or row-filtering operation and supply its parameter.
3. Optionally define condition column, operator and comparison value.
4. Validate required inputs and check column/operation compatibility.
5. Create a copy of the original DataFrame.
6. Build a Boolean mask if a condition is provided; otherwise select all applicable rows.
7. Apply the selected operation to the chosen values or records.
8. Create a named scenario and record its metadata.

9. Refresh results and make the scenario available for comparison or export.

9.2 Pseudocode: Experiment Execution

```
INPUT: original_data, target_column, operation, value, optional_condition
VALIDATE required inputs
scenario_data = COPY(original_data)
IF condition exists:
    mask = EVALUATE_CONDITION(scenario_data)
ELSE:
    mask = SELECT_ALL_APPLICABLE_ROWS
APPLY operation using mask
STORE scenario name and metadata
RETURN scenario_data
```

9.3 Input Validation

Validation is important because the application accepts user-supplied file paths, column names, operators, and values. Checks should confirm that a dataset is loaded, the target column exists, numerical operations use compatible values, and required inputs can be interpreted. Invalid input should produce a clear message.

10. DATASET HANDLING AND VALIDATION

10.1 Dataset Loading

The user may load the included sample CSV or select an external CSV file. Pandas reads the file and represents it as a DataFrame. The DataFrame provides labelled rows and columns that can be inspected and transformed.

10.2 Sample Dataset

The included demonstration dataset contains 12 student-related records and numerical fields such as Study_Hours, Attendance, and Marks. It is illustrative, not real institutional data. Other CSV files may be loaded if their structure is compatible with the selected operations.

10.3 Data Inspection

- Inspect column names and record previews.
- Identify numerical columns for arithmetic operations and mean comparisons.
- Check whether the selected column contains compatible values.

- Review missing or non-numeric values before interpreting results.
- Confirm common columns when comparing a second dataset.

10.4 Validation Considerations

CSV files may contain missing values, mixed data types, duplicate records, or inconsistent column names. These conditions can affect calculations. The application should handle common errors with user-friendly messages. Not every arbitrary CSV file is suitable for every operation.

10.5 Pseudocode: Loading a Dataset

```
INPUT: selected_file_path
IF file path is valid:
    dataset = READ_CSV(selected_file_path)
    INSPECT columns and data types
    DISPLAY preview and summary
ELSE:
    DISPLAY file-loading error
```

11. COMPARISON AND STATISTICAL ANALYSIS

The Comparison Studio compares the original dataset with a saved scenario or a second CSV dataset. The comparison is based on common numerical columns so that the same type of measurement can be examined in both datasets.

11.1 Comparison Procedure

10. Select original dataset and scenario or second CSV.
11. Identify columns present in both datasets.
12. Determine which common columns are numerical.
13. Calculate the mean of each selected column in both datasets.
14. Calculate absolute and percentage change where defined.
15. Display numerical summary and grouped bar chart.

11.2 Mean

The arithmetic mean is calculated by adding numerical observations and dividing by the number of valid observations. It provides a simple summary of the central value of a numerical column.

11.3 Absolute Change

Absolute Change = Modified Mean − Original Mean. This value shows the difference in the original units of the column. A positive value indicates an increase in the mean, while a negative value indicates a decrease.

11.4 Percentage Change

Percentage Change = (Absolute Change ÷ Original Mean) × 100. This expresses the difference relative to the original mean. If the original mean is zero, the formula is undefined and the application should avoid presenting a misleading percentage.

11.5 Worked Example

Table 11. Example of Mean Comparison

Measure	Value
Original Study_Hours mean	3.38
Modified Study_Hours mean	3.90
Absolute change	3.90 − 3.38 = 0.52
Percentage change	(0.52 ÷ 3.38) × 100 ≈ 15.38%

This example explains the calculation only. Replace it with actual application output if reporting a real experiment.

11.6 Pseudocode: Comparison

```
common_columns = INTERSECTION(columns_A, columns_B)
numeric_columns = COMMON_NUMERIC_COLUMNS(common_columns)
FOR each column:
    mean_A = MEAN(dataset_A[column])
    mean_B = MEAN(dataset_B[column])
    absolute_change = mean_B - mean_A
    IF mean_A != 0:
        percentage_change = absolute_change / mean_A * 100
    ELSE:
        percentage_change = NOT_DEFINED
DISPLAY summary and chart
```

12. DATA VISUALISATION

Visualisation helps users interpret numerical comparisons quickly. The application uses Matplotlib to create a grouped bar chart. For each common numerical column, the chart displays one bar for the original mean and another for the modified mean.

Figure 9. Illustrative Comparison of Mean Values

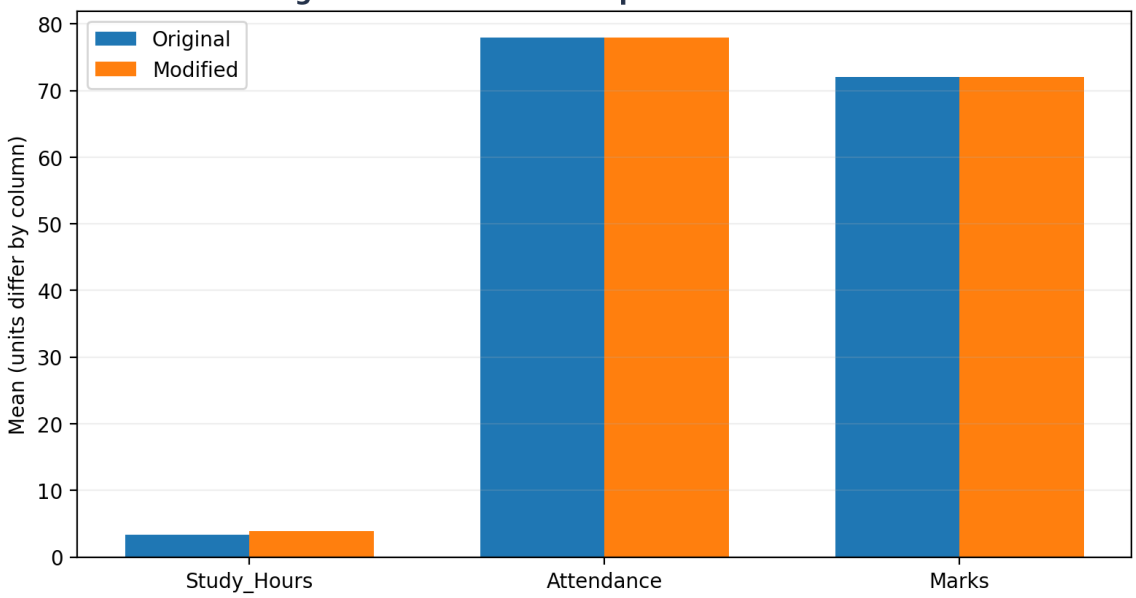


Figure 9. Illustrative grouped bar chart; values are examples, not verified application output.

12.1 Reading the Chart

The horizontal axis lists selected numerical columns, while the vertical axis represents their mean values. A legend distinguishes the original dataset from the modified dataset. Since different columns may use different units or scales, the chart should be interpreted carefully. Bar heights are most useful for within-column comparison.

12.2 Benefits

- Makes differences between original and modified means easier to see.
- Complements numerical summaries.
- Provides a compact visual representation for reports.
- Helps identify which selected columns changed after an experiment.

12.3 Limitations of the Chart

A mean comparison does not show the full distribution of data. It may hide outliers, variation, or changes in individual records. Histograms or box plots could be added in a future version.

13. INSIGHTS AND REPORT GENERATION

13.1 Descriptive Insights

The Insights & Reports module presents observations derived from calculated summary values. For example, it may state that a column's mean increased or decreased and show the corresponding difference. Such statements describe the comparison and do not establish why the change occurred.

13.2 Exporting a Modified Dataset

The user can export the active modified dataset to a CSV file. This allows the scenario to be opened in a spreadsheet or another data-analysis environment. The exported file should be named clearly so that it can be distinguished from the original.

13.3 Exporting a Text Report

The text report provides a portable summary of the experiment and comparison. Depending on the information available, it may include scenario name, selected operation, target column, comparison statistics, and descriptive observations.

13.4 Pseudocode: Export

```
INPUT: active_scenario, report_summary
ASK user for output location
WRITE active_scenario to CSV
FORMAT report_summary as readable text
WRITE summary to text file
CONFIRM export
```

13.5 Experiment History

The project uses JSON for experiment-history metadata. This can record descriptive information about saved experiments, such as names and parameters. JSON is a lightweight file format and is not a relational database.

14. SYSTEM WORKFLOW AND DATA FLOW

Figure 2. Overall System Workflow

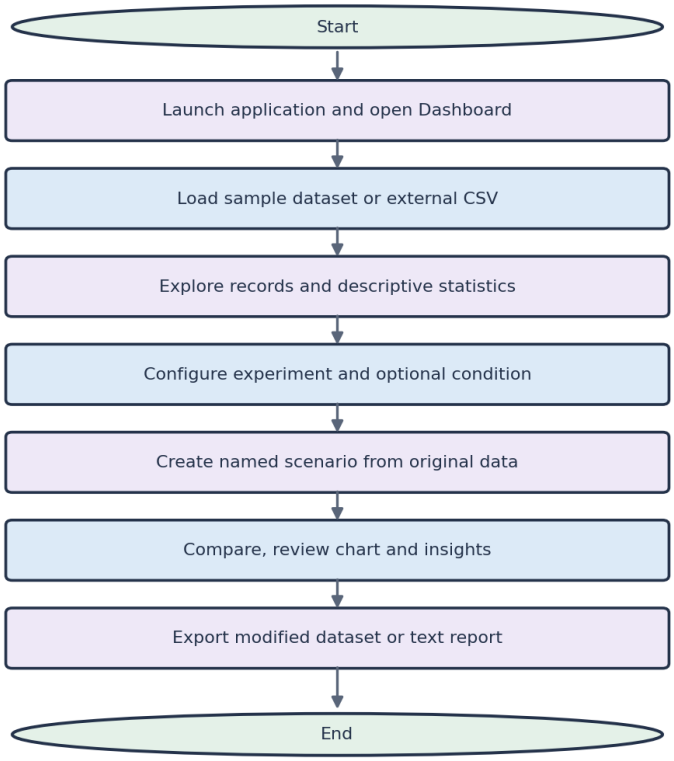


Figure 2. Overall workflow of the application

Figure 5. Data Processing Workflow

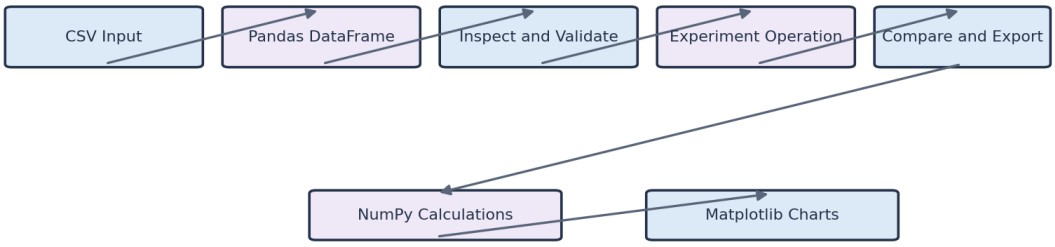


Figure 5. Data processing workflow

The workflow begins when the user launches the application and loads a sample or external CSV dataset. After inspecting the data, the user configures an experiment, creates a named scenario, and opens the comparison function. The application calculates descriptive differences and produces a chart. Finally, the user can review insights and export the modified dataset or report.

14.1 Use Case Diagram

Figure 6. Use Case Diagram – User Functions

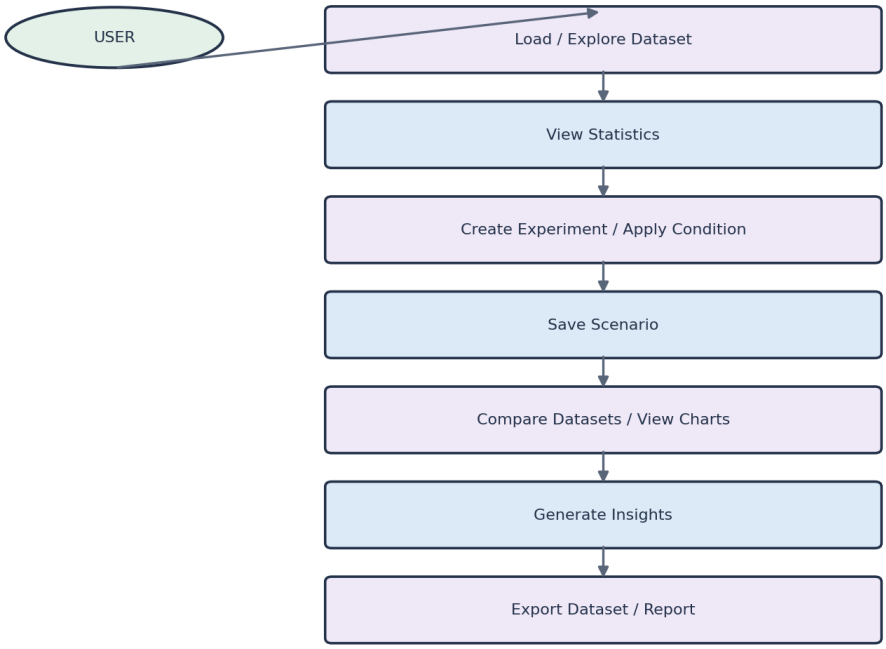


Figure 6. User interactions represented as a use case diagram

The user is the primary actor. The use cases represent the main actions available in the application: loading and exploring data, creating experiments, saving scenarios, comparing datasets, viewing charts, generating insights, and exporting results.

15. DATA FLOW DIAGRAMS

15.1 DFD Level 0

The Level 0 Data Flow Diagram treats WHAT IF? as one overall process. The user supplies a CSV dataset, experiment parameters, and optionally a comparison dataset. The application produces a modified dataset, comparison statistics, visual charts, and a text report.

Figure 7. DFD Level 0

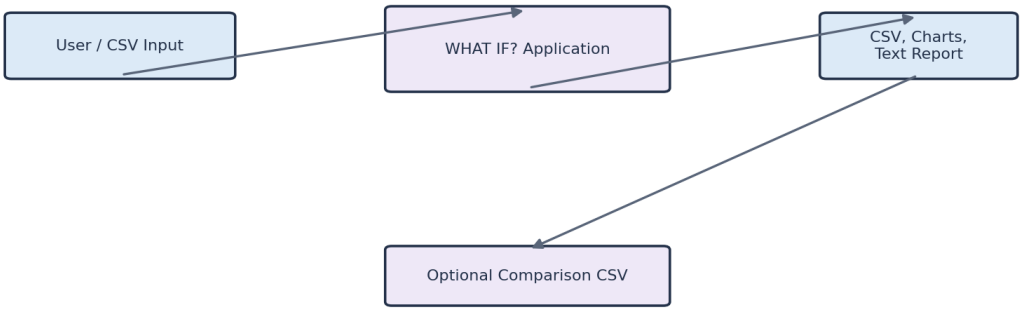


Figure 7. Data Flow Diagram – Level 0

15.2 DFD Level 1

The Level 1 diagram decomposes the main process into dataset management, experiment processing, scenario management, dataset comparison, and insights/export. CSV files act as input and output data stores, while JSON stores experiment metadata.

Figure 8. DFD Level 1

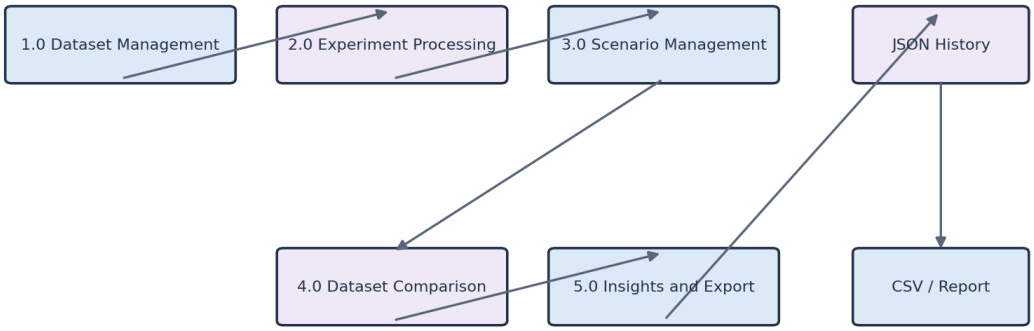


Figure 8. Data Flow Diagram – Level 1

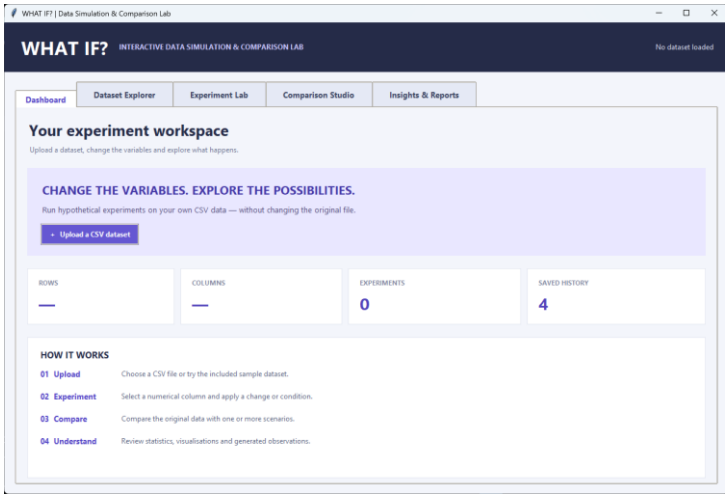
16. USER INTERFACE DESIGN

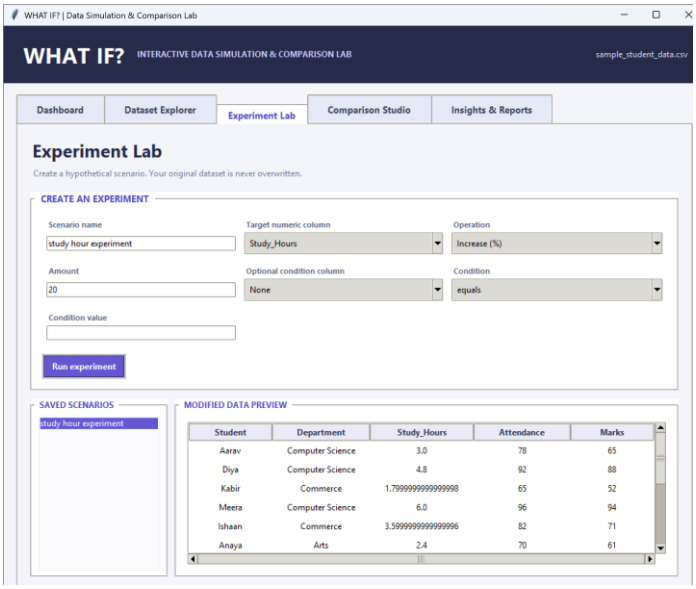
The user interface is built with Tkinter and is organised into separate functional areas. The Dashboard provides navigation, while Dataset Explorer, Experiment Lab, Comparison Studio, and Insights & Reports screens focus on distinct tasks. This separation helps users follow a logical workflow and reduces the number of controls shown at one time.

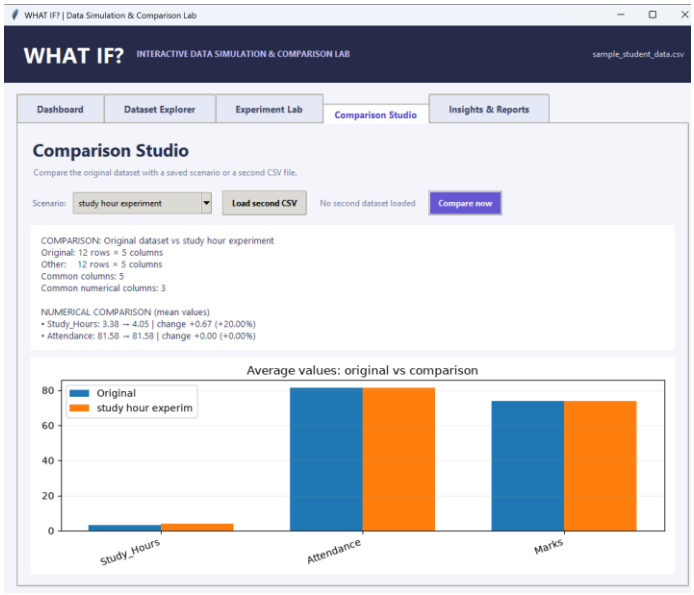
16.1 Interface Design Principles

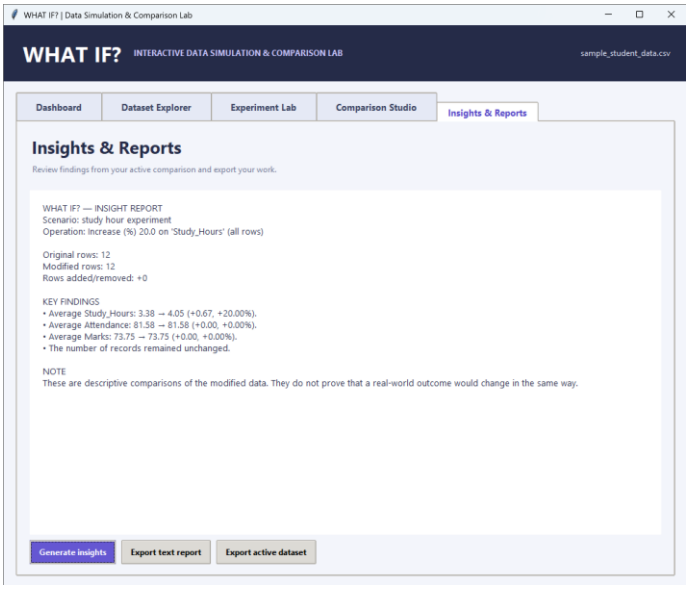
- Consistency: similar controls should behave in similar ways.
- Clarity: labels should describe the selected operation and expected input.
- Feedback: the application should show status messages and validation errors.
- Separation: exploration, experimentation, comparison, and export remain distinct tasks.
- Readability: tables, chart labels, and result summaries should be easy to interpret.

16.2 Screenshots









17. IMPLEMENTATION DETAILS

17.1 Application Structure

The project is organised around a main Python file, dependency information, documentation, a Windows launch helper, and a data directory containing the sample CSV. The main application logic is in app.py. The requirements file lists external Python packages needed to run the application.

Table 12. Project Folder Structure

Path	Purpose
WhatIf_Data_Simulation_Lab/	Root project directory.
app.py	Main Python application and GUI logic.
requirements.txt	Lists Python package dependencies.
README.md	Project overview and setup instructions.
.gitignore	Specifies files Git should ignore.
run.bat	Optional Windows batch launcher.
data/	Contains demonstration dataset.
data/sample_student_data.csv	12-record sample dataset.

17.2 Dataset Representation

After loading, a CSV dataset is represented as a Pandas DataFrame. Column labels allow the application to refer to a target variable by name. Boolean masks identify rows satisfying a condition, and DataFrame operations update selected values or filter records.

17.3 Scenario Handling

A scenario is a modified version of the original DataFrame created using the user's selected operation. Retaining the original baseline allows independent scenarios to be compared consistently. The scenario name and related metadata can be recorded in JSON.

17.4 Error Handling

The application should handle common errors such as missing files, invalid numeric inputs, unavailable columns, incompatible data types, and comparisons with no common numerical columns. Error messages should explain the issue in plain language.

18. TESTING AND VALIDATION

Testing checks whether the application's major workflows behave as expected. The following table provides a practical test plan. The actual result and status must be completed after running each test in the final application; they are deliberately not marked as passed without verification.

Table 13. Proposed Test Cases

ID	Test Scenario	Expected Result	Actual Result / Status
TC-01	Launch application	Main window opens.	To be verified
TC-02	Load sample CSV	Sample records appear.	To be verified
TC-03	Load external CSV	Compatible CSV is loaded.	To be verified
TC-04	Increase column by 20%	Selected values increase correctly.	To be verified
TC-05	Decrease column by 10%	Selected values decrease correctly.	To be verified
TC-06	Apply conditional operation	Only matching rows are modified.	To be verified
TC-07	Keep matching rows	Only matching records remain.	To be verified
TC-08	Remove matching rows	Matching records are removed.	To be verified
TC-09	Compare original and scenario	Common numerical statistics display.	To be verified
TC-10	Generate comparison chart	Grouped bar chart appears.	To be verified
TC-11	Export modified dataset	CSV file is written.	To be verified
TC-12	Export text report	Readable text report is created.	To be verified

18.1 Validation Scenarios

In addition to normal use, test invalid inputs: an unavailable file, a missing target column, non-numeric input for arithmetic, a condition that matches no records, and datasets with no common numerical columns. The application should respond with clear feedback.

18.2 Testing Limitations

This report provides a proposed test plan, not a claim that all tests have been executed. Update the status column with the observed outcome after testing the final version.

19. RESULTS AND DISCUSSION

The intended result is a desktop environment in which users can load and inspect a dataset, apply a selected transformation, create a named scenario, compare that scenario with a baseline, and export the result. The project demonstrates an introductory data workflow: input, inspection, transformation, summary, visualisation, and output.

The Experiment Lab demonstrates how a mathematical operation or condition can change a dataset. The Comparison Studio demonstrates how summary statistics can describe differences between two data versions. The grouped bar chart complements the numerical summary and makes selected changes easier to observe.

The project demonstrates the integration of several Python libraries. Tkinter handles interaction, Pandas manages tabular data, NumPy supports numerical work, and Matplotlib presents the results. CSV and JSON provide lightweight file-based storage for datasets and metadata.

The quality of results depends on input data quality and experiment parameters. A comparison of means is only one summary of the data and should not be treated as a complete statistical evaluation.

20. LIMITATIONS AND FUTURE ENHANCEMENTS

20.1 Current Limitations

- The primary dataset format is CSV; other formats are outside the stated current scope.
- The application focuses on basic transformations and descriptive statistics.
- It does not include predictive modelling or causal analysis.
- Results depend on input data quality and compatible column types.
- Percentage change is undefined when the original mean is zero.
- A mean-based chart does not show the full distribution.
- The current application is designed for individual desktop use.

20.2 Future Enhancements

Table 14. Limitations and Future Enhancements

Current Limitation	Possible Future Enhancement
CSV-focused import	Add Excel and other structured file formats.
Basic charting	Add histograms, line charts, box plots and scatter plots.
Limited statistics	Add median, standard deviation, quartiles and correlation.
No undo/redo	Introduce reversible experiment actions.
Basic scenario organisation	Add scenario groups, duplicate and rename controls.
Simple filters	Support combined conditions and more operators.
Basic report export	Add formatted PDF and DOCX report generation.
Individual desktop workflow	Consider controlled sharing or multi-user support.

21. CONCLUSION

WHAT IF? – Interactive Data Simulation and Comparison Lab demonstrates how a desktop application can make basic data experimentation more accessible. By combining dataset exploration, hypothetical modifications, scenario creation, statistical comparison, visualisation, and export, the project provides a structured workflow for asking and examining simple 'what if' questions.

The project applies Python and its data-science ecosystem in a practical way. Tkinter provides the graphical interface, Pandas supports DataFrame-based processing, NumPy assists numerical operations, and Matplotlib creates comparison charts. CSV and JSON provide straightforward file formats for datasets and experiment metadata.

A key design idea is to preserve the original dataset as a baseline and create scenarios from that baseline. This supports consistent comparisons and makes the effect of each selected operation easier to inspect. The Comparison Studio summarises changes using mean values, absolute differences, and percentage differences where appropriate.

The application is educational and descriptive. It shows the result of explicitly defined transformations and does not predict future behaviour or prove causation. Within that scope, the project provides practical experience in GUI development, data handling, conditional logic, elementary statistics, charting, file operations, and software organisation.

21.1 Learning Outcomes

- Experience designing a multi-section Tkinter desktop application.
- Practice loading and manipulating CSV data with Pandas.
- Apply conditions and arithmetic transformations to DataFrames.
- Calculate and interpret means and percentage changes.
- Create charts with Matplotlib.
- Organise a small Python project and document its functionality.

22. REFERENCES

The following official documentation sources are appropriate references for the technologies used. Consult the current pages when preparing the final submission and follow the citation style required by the institution.

16. Python Software Foundation. Python Documentation. https://docs.python.org/3/
17. Python Software Foundation. tkinter – Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html
18. The pandas development team. pandas documentation. https://pandas.pydata.org/docs/
19. NumPy Developers. NumPy documentation. https://numpy.org/doc/
20. Matplotlib Development Team. Matplotlib documentation. https://matplotlib.org/stable/

23. APPENDIX

Appendix A: Application Execution Steps

21. Install a compatible Python 3 version.
22. Open the project folder in Visual Studio Code or a terminal.
23. Create and activate a virtual environment if required.
24. Install dependencies listed in requirements.txt using pip.
25. Run the application using python app.py, or use the Windows launcher if configured.
26. Load the sample CSV or select a compatible external CSV file.
27. Explore the dataset, create a scenario, compare results, and export outputs.

Appendix B: Example Experiment

Experiment name: Increased Study Hours. Target column: Study_Hours. Operation: Increase (%). Value: 20. Optional condition: Attendance greater than 75. Expected behaviour: the application creates a scenario in which selected Study_Hours values for matching records are increased by 20%, while the original dataset remains the baseline.